

Notas de Conferencia.

Lenguaje de programación Swift.

Ponentes: German Santos Jaimes

Marduk Pérez de Lara Domínguez

Datos clave

Lanzamiento. 2014 - desarrollo en secreto durante 4 años por Apple.

Creador. Chris Lattner.

Plataformas. iOS · macOS · watchOS · tvOS · Linux · Windows · Android (SDK)

Paradigma. Multi-paradigma, imperativo, orientado a objetos, funcional, reactivo.

Junior en ... Aproximadamente 6 meses de estudio intensivo para nivel competitivo del mercado.

Origen de Swift

→ **Nombre y símbolo.** Lleva el nombre del pájaro vencejo debido a sus cualidades: pequeño, ligero, rápido y ágil. El logo representa al ave en posición de picada como metáfora directa del rendimiento que tiene como objetivo el lenguaje.

→ **Lanzamiento.**

- * Lanzado en la WWDC 2014 tras cuatro años de desarrollo en secreto.
- * El equipo de desarrollo fue liderado por Chris Lattner, creador además de LLVM.
- * Lattner trabajó posteriormente en Tesla y en el equipo de TensorFlow en Google.

→ **Evolución del lenguaje**

- * Es una evolución de C y Objective-C.
- * Tiene una sintaxis inspirada en Python, con la potencia de un lenguaje compilado.
- * Para garantizar un uso de memoria óptimo y seguro combina la inferencia de tipos y la compilación.

Cómputo móvil

- * App Store (2008) impulsó el modelo de app enfocada en una única función: pedir comida, transporte, depósitos bancarios.
- * Contrasta con el software monolítico tradicional que intentaba resolver todo.
- * Batería, memoria RAM limitada y conectividad intermitente son los grandes retos del desarrollo móvil.
- * Swift fue diseñado específicamente para ser eficiente en estos entornos restringidos en hardware.
- * La transición de Intel (CISC) a ARM (RISC) en los chips Apple reduce el consumo energético y generación de calor.
- * Apple controla el stack dado que tiene hardware propio (chips serie A y M con arquitectura ARM), tiene el sistema operativo (iOS, macOS) y el lenguaje de programación (Swift) y frameworks (SwiftUI, Foundation).

Innovaciones tecnológicas.

→ SwiftUI. Framework declarativo

- * Permite crear interfaces de usuario con mucho menos código.
- * Live Previews. Permite visualizar cambios en tiempo real sin recompilar constantemente.
- * Migración del modelo imperativo (UIKit: le dices paso a paso qué hacer) al declarativo (declaras cómo debe verse la UI).

→ Gestión de memoria

- * Automatic Reference Counting elimina la gestión manual de apuntadores heredada de Objective-C.
- * Permite velocidad comparable a C++ con la facilidad de uso de lenguajes de alto nivel.

→ Concurrency

- * Swift incorporó manejo nativo de tareas asíncronas y concurrentes con async/await.
- * vital para apps que dependen de servicios en la nube y procesan datos en segundo plano.

→ Programación reactiva

- * Vista y modelos de datos siempre sincronizados.
- * Si un dato cambia en la base de datos, la interfaz se actualiza automáticamente.

→ Multiplataforma

- * SDK de Swift para Android.
- * Compilación en Linux y Windows.
- * Un mismo programador puede escribir cliente (móvil) y servidor (nube) en el mismo lenguaje.

Oportunidades académicas y profesionales

→ Swift Playgrounds.

- * Disponible en iPad y Mac, no es solo para niños.
- * Entorno de desarrollo: permite prototipar algoritmos y ver resultados gráficamente.
- * Es posible subir aplicaciones directamente a la App Store desde un iPad.

→ Mercado laboral

- * Se estima que con 6 meses de estudio intensivo se puede alcanzar nivel Junior.
- * Existe la tendencia creciente hacia el desarrollo nativo sobre el híbrido por rendimiento y UX.

→ Becas, academias y convenios con la UNAM

- * Swift Student Challenge: competencia anual de Apple para estudiantes.
- * Academia Swift en Italia con becas disponibles para estudiantes latinoamericanos.
- * iOS Lab en la FI. Equipos y capacitación especializada.

Conclusión

Lo que me queda después de esta conferencia no es solo una lista de características técnicas, sino una forma de ver el desarrollo de software donde las restricciones no son un enemigo pueden ser el punto de partida pues Swift nació de limitaciones reales y las convirtió en sus principios. Swift considera que el rendimiento debe ser la idea central. No había visto la magnitud de la importancia de que Apple controle el stack completo desde los chips hasta el framework SwiftUI pues ha creado un entorno donde un solo lenguaje puede resolver desde la interfaz de usuario hasta el backend en la nube.

Tampoco había entendido que Swift representa una de las rutas más claras y bien documentadas para el mercado laboral, con un estimado de seis meses para alcanzar un nivel competitivo.

